Od zera do pierwszego programu

LEKCJA 0 · PODSTAWY PROGRAMOWANIA

Jak komputer zapisuje liczby, litery i kolory, czym jest algorytm i jak napisać pierwszy program w Scratchu.

Plan zajęć

CZAS	TEMAT
0–5 min	o kursie
5–13 min	AI i programowanie
13–30 min	wejście i wyjście, system jedynkowy i dwójkowy
30–48 min	litery, kolory i media jako liczby
48–68 min	algorytmy i pseudokod
68–86 min	Scratch na żywo
86–90 min	zadanie domowe

Trzy pytania o każdy program

1 · POPRAWNOŚĆ

Czy robi to, co miał robić, także w nietypowych sytuacjach?

2 · PROJEKT

Czy robi to rozsądnie: bez powtórzeń, bez marnowania pamięci i czasu?

3 · STYL

Czy łatwo go przeczytać: nazwy, porządek, komentarze, wcięcia?

AI i programowanie

AI pisze kod szybciej niż człowiek. Ocenić go, opisać problem i wyłapać pułapki musisz jednak ty.

Dlatego zaczynamy od podstaw.

Po co programowanie, skoro AI pisze kod?

OCENA

Bez rozumienia kodu nie ocenisz, czy jest poprawny i bezpieczny.

SPECYFIKACJA

Opisanie problemu krok po kroku zostaje po stronie człowieka.

PUŁAPKI

Przepełnienia i zaokrąglenia AI potrafi przemycić po cichu.

NAUKA

Trening myślenia, które rozwiązuje złożone problemy.

Prompt: co wysyłasz do modelu

	PROMPT SYSTEMOWY	PROMPT UŻYTKOWNIKA
co to jest	instrukcja „za kulisami”: rola, ton i zasady	konkretna prośba
kto go pisze	twórca aplikacji	osoba, która z niej korzysta
przykład	„Jesteś copywriterem. Odpowiadaj jednym hasłem.”	„Hasło na plakat festiwalu animacji”

Prompt systemowy działa jak **brief** dla współpracownika.

Pokaz na żywo: ten sam brief, dwa prompty

WEJŚCIE

„Zaproponuj hasło na plakat festiwalu filmów animowanych”

→

CZARNA SKRZYNKA

Claude z promptem systemowym i bez niego

→

WYJŚCIE

dwa różne hasła

Pytanie do sali: co zmieniło jedno zdanie w prompcie systemowym?

Zasada na lekcjach 0–2

AI może **tłumaczyć** pojęcia i błędy, ale **nie odrabia** za ciebie zadań.

Tryb kaczki: AI naprowadza pytaniami, jak dobry korepetytor.

Wejście, czarna skrzynka, wyjście

WEJŚCIE

zdjęcie produktu na białym tle

→

CZARNA SKRZYNKA

narzędzie do usuwania tła

→

WYJŚCIE

plik PNG z przezroczystym tłem

Każdy program da się tak opisać. **Czarna skrzynka**: znasz wejście i wyjście, ale nie musisz znać środka.

Reprezentacja danych

Komputer ma tylko prąd: płynie albo nie płynie.

Z tego zbudujemy liczby, litery, kolory, dźwięk i obraz.

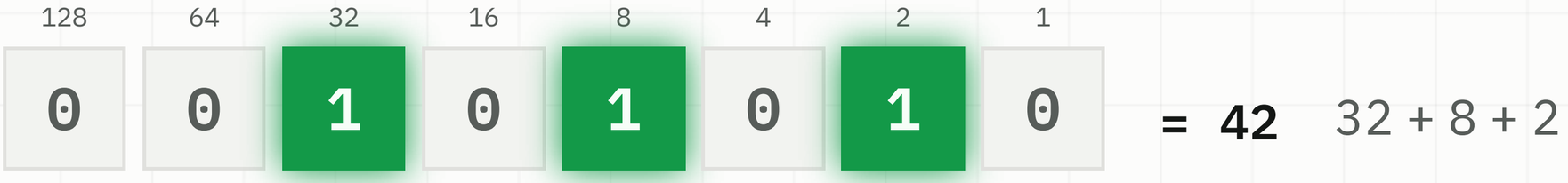
System jedynkowy: jeden palec, jedna osoba

- 5 osób to 5 kresek, a 1000 osób to **1000 kresek**
- zapis rośnie **liniowo**: dwa razy większa liczba to dwa razy więcej kresek
- na dwóch rękach policzysz tylko do **10**

> **Na zajęciach:** na ile osób starczy wam rąk?

System dwójkowy: liczba z żarówek

Każde miejsce ma **wagę**: 1, 2, 4, 8, 16… Sumujemy wagi zapalonych żarówek.



= 42 32 + 8 + 2

> **Na zajęciach:** 31 na jednej ręce. Do ilu policzysz na dwóch?

Bit i bajt

ILE BITÓW	ILE WARTOŚCI	ZAKRES
1 bit	2	0 albo 1
8 bitów, czyli **bajt**	256	od 0 do 255
16 bitów	65 536	od 0 do 65 535
n bitów	2^n	od 0 do $2^n - 1$

Każdy dodatkowy bit **podwaja** zakres.

Na zajęciach: żywe żarówki

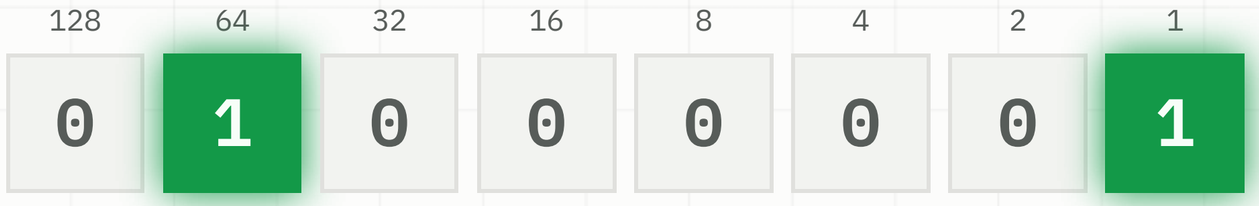
Osiem osób to jeden bajt. Ustawcie liczbę 72.

128	64	32	16	8	4	2	1
0	1	0	0	1	0	0	0

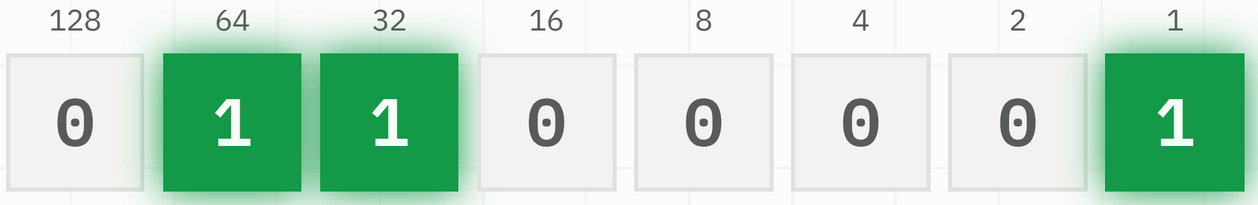
= 72

ASCII: liczby, które są literami

Umowa: **65 to A**, 66 to B, 97 to a. Razem 128 kodów.



= **65** litera A



= **97** litera a

Wielkość litery zmienia **jeden bit**, ten o wadze 32.

Zagadka

Co znaczy `72 73 33`?

Odpowiedź

`72 73 33` to napis **HI!**

H to 72, I to 73, a wykrzyknik to 33.

Unicode i UTF-8

Unicode daje każdemu znakowi numer. **UTF-8** zapisuje ten numer w bajtach.

ZNAK	NUMER W UNICODE	BAJTY W UTF-8
A	U+0041	1
ą	U+0105	2
中	U+4E2D	3
😂	U+1F602	4

Skąd się biorą krzaczki?

Litera `ą` zapisana w UTF-8 i odczytana według starej umowy Windows-1250 zmienia się w `Ä…`. **Bajty są w porządku, zła jest umowa odczytu.**

To samo dzieje się z całymi plikami tekstowymi i napisami na stronach.

Kolor to trzy liczby

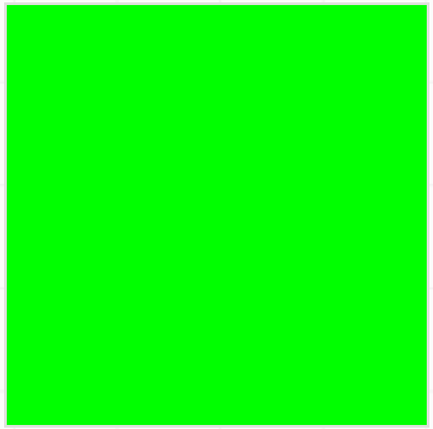
Każdy piksel to trzy bajty: ile **czerwieni**, **zieleni** i **niebieskiego**, od 0 do 255.



#FF5733

`rgb(255, 87, 51)`

pomarańczowo-czerwony



#00FF00

`rgb(0, 255, 0)`

czysta zieleń



#000000

`rgb(0, 0, 0)`

czarny

#FFFFFF

`rgb(255, 255, 255)`

biały



#484921

`rgb(72, 73, 33)`

„HI!” zapisane jako kolor

Ile to waży?

DANE	WYNIK
jedna klatka Full HD: 1920 × 1080 × 3 bajty	≈ **6,2 MB**
1 sekunda wideo, 30 klatek	≈ **186,6 MB**
1 minuta wideo	≈ **11,2 GB**

Dlatego potrzebna jest **kompresja**: bezstratna (PNG, ZIP, FLAC) albo stratna (JPEG, MP3, H.264).

Najważniejsza idea lekcji

`72 73 33` to liczby, napis albo kolor.

Wszystko zależy od umowy.

W języku C taką umowę nazywamy typem danych.

Na zajęciach: zgadnij liczbę

Myślę o liczbie od 1 do 1000. Ile pytań „więcej czy mniej” wystarczy?

Odpowiedź: 10 pytań

Każde pytanie o środek odrzuca **połowę** możliwości:

```
1000 → 500 → 250 → 125 → 63 → 32 → 16 → 8 → 4 → 2 → 1
```

$2^{10} = 1024$, więc 10 pytań zawsze wystarczy.

Trzy sposoby szukania w książce telefonicznej

STRONA PO STRONIE

Do **n** kroków. Przy milionie stron: milion kroków.

PO DWIE STRONY

Około **n/2** kroków, ale bez poprawki może przeskoczyć nazwisko.

OD ŚRODKA

Około **$\log_2 n$** kroków. Przy milionie stron: **20** kroków.

Pseudokod: algorytm zwykłym językiem

```
1  Weź książkę telefoniczną
2  Otwórz na środku książki
3  Spójrz na stronę
4  Jeśli osoba jest na stronie
5      Zadzwoń do osoby
6  W przeciwnym razie, jeśli osoba jest wcześniej w książce
7      Otwórz na środku lewej połowy książki
8      Wróć do linii 3
9  W przeciwnym razie, jeśli osoba jest później w książce
10     Otwórz na środku prawej połowy książki
11     Wróć do linii 3
12 W przeciwnym razie
13     Zakończ
```

Cztery klocki każdego programu

W PSEUDOKODZIE	POJĘCIE
„Otwórz”, „Spójrz”, „Zadzwoń”	**funkcja**: czynność
„Jeśli… w przeciwnym razie…”	**warunek**: rozgałęzienie
„osoba jest na stronie”	**wyrażenie logiczne**: pytanie, na które odpowiedź to tak albo nie
„Wróć do linii 3”	**pętla**: powtórzenie

Na zajęciach: rysunek pod dyktando

Ktoś dyktuje rysunek z figur, a prowadzący **rysuje dokładnie to, co słyszy.**

Komputer robi to, co mu każemy, a nie to, co mieliśmy na myśli.

Scratch: programowanie klockami

KLOCEK	POJĘCIE
`when green flag clicked`	**zdarzenie**: tu zaczyna się skrypt
`say [Witaj, świecie!]`	**funkcja** z **argumentem**
dymek nad duszkiem	**efekt uboczny**
`join [Cześć, ] (answer)`	**wartość zwracana**
`repeat (10)`, `forever`	**pętla**
`if < > then … else`	**warunek**

Scratch na żywo

1 · WITAJ, ŚWIECIE

`when green flag clicked` i `say`

2 · WITAJ, TY

`ask` , `answer` i `join`

3 · MIAU

`repeat` i własny blok `miau (n) razy`

4 · WARUNKI

`if` i `touching (mouse-pointer)?`

Od Scratcha do C

```c
#include <stdio.h>

int main(void)
{
    printf("hello, world\n");
}
```

`main` działa jak `when green flag clicked`, a `printf` jak `say`. Tym zajmiemy się na lekcji 1.

Zadanie domowe: projekt w Scratchu

Historia, gra albo animacja, np. animowane logo albo interaktywny plakat.

- co najmniej **dwa duszki**, w tym jeden, który nie jest kotem
- co najmniej **trzy skrypty**
- **warunek**, **pętla** i **zmienna**
- **własny blok** z parametrem
- tekst z **polskimi znakami** i plik `README.txt` w UTF-8

Oddajesz link do projektu i plik `.sb3`.



Pytania?

DO ZOBACZENIA NA LEKCJI 1

Cały materiał, laboratoria i quizy znajdziesz w kursie, w lekcji 0. Na następnych zajęciach: język C.